第二次上机实验题目

1、 循环程序设计

数据段中已定义了一个长度为 n（n>=16）的字数组 M，试编写一程序求出 M 中绝对值最大的数，把它放在数据段的 M+2n 单元中，并将该数的偏移地址存放在 M+2(n+1)单元中。要求：

（1）把自己学号每 2 位一组，分别定义前 5 个字的初始值。例如，某同学的学号为 2195012345，则数组 M 的前 5 个字的定义为：21H， 95H，01H，23H，45H，剩余的字根据程序调试需要进行初始化。

2、 分支程序设计

设计一个程序，在数据段用自己的学号定义一个字符串，字符串长度大于 20，对该字符串中字符 0~9 字符出现的次数进行计数，计数结果存放在以 COUNT 为首地址的数组中，并将出现次数最多的字符和出现次数显示出来。要求：

（1）该字符串的前 10 个字符必须为自己的学号，例如某同学的学号为 2195012345，则该字符串的定义为 '2195012345xxxxxxxxxxxxxxxx'，字符串中的 x 根据程序调试需要进行初始化；

（2）如果字符 9 出现的次数为 5 次，且是出现次数最多的字符，则在程序结束时输出： 9，5。若有多个次数相同的字符，则输出字符最大的哪个。

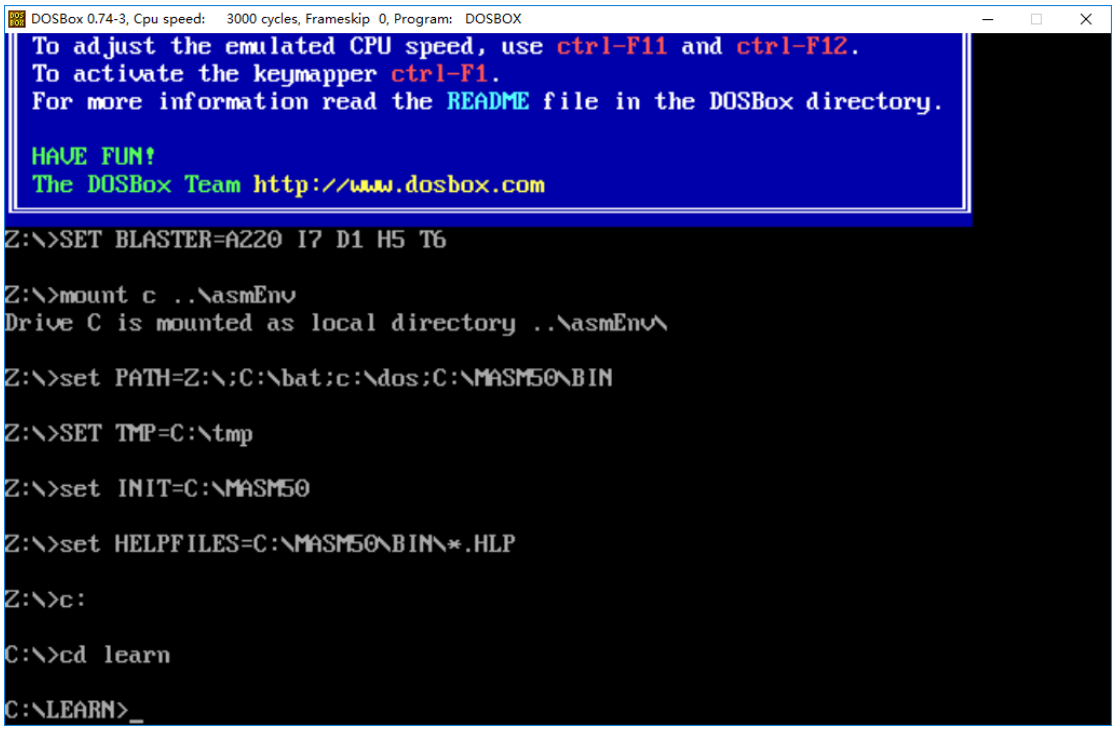
实验要求：

1、 在上机时间内，完成上述实验内容，并按要求在思源学堂**提交上机结果**，具体格式和要求见 **提交上机结果的模板文件**；

2、 **上机结果文件转换为 PDF 格式后再进行提交。**

截图说明：

在 windows 下使用 Alt+Print 组合键可以对当前窗口进行截图，然后可以直接粘贴在这个文档中。这里以上机环境刚启动时为例进行说明，上机环境启动时的截图：



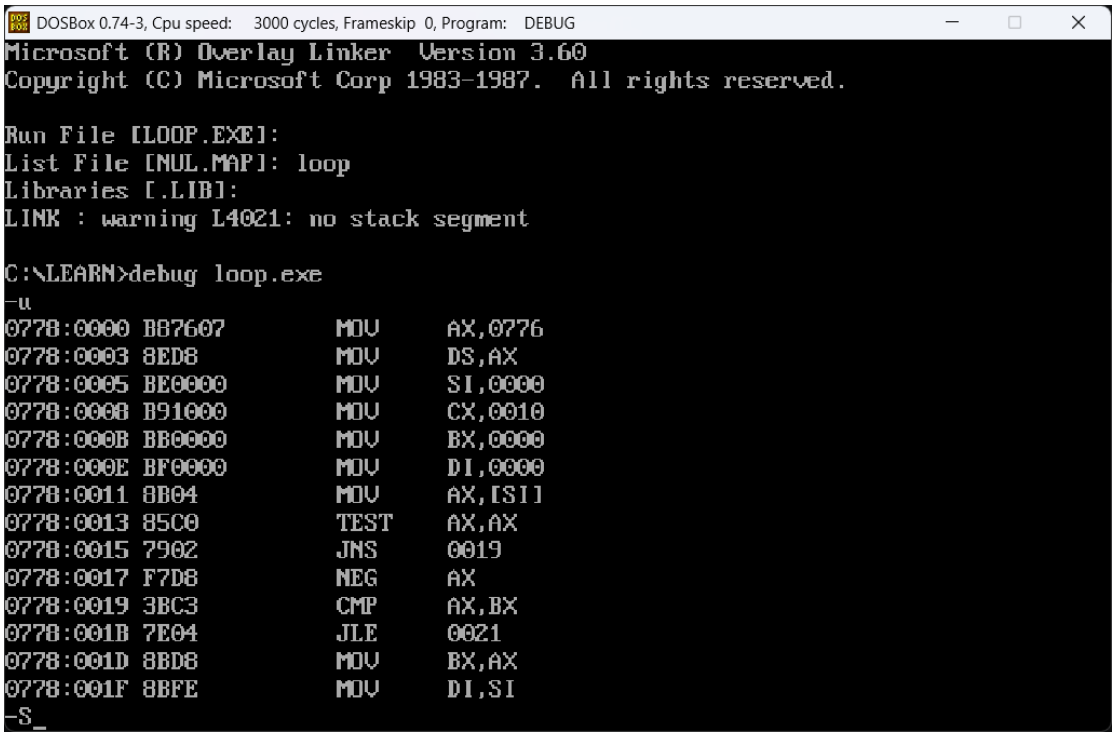
提交上机结果的模板文件

第 2 次上机

班级	学号	姓名
计算机 2301	2234412799	李祥熙

1、 循环程序设计

（1）反汇编的截图



```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0778:000E BF0000        MOV     DI,0000
0778:0011 8B04          MOV     AX,[SI]
0778:0013 85C0          TEST    AX,AX
0778:0015 7902          JNS     0019
0778:0017 F7D8          NEG     AX
0778:0019 3BC3          CMP     AX,BX
0778:001B 7E04          JLE     0021
0778:001D 8BD8          MOV     BX,AX
0778:001F 8BFE          MOV     DI,SI
-u
0778:0021 83C602        ADD     SI,+02
0778:0024 E2EB          LOOP    0011
0778:0026 BE0000        MOV     SI,0000
0778:0029 83C620        ADD     SI,+20
0778:002C 891C          MOV     [SI],BX
0778:002E 83C604        ADD     SI,+04
0778:0031 893C          MOV     [SI],DI
0778:0033 B8004C        MOV     AX,4C00
0778:0036 CD21          INT     21
0778:0038 8A86FAFE      MOV     AL,[BP+FEFA]
0778:003C 2AE4          SUB     AH,AH
0778:003E 40            INC     AX
0778:003F 50            PUSH    AX
0778:0040 8D86FAFE      LEA     AX,[BP+FEFA]
-S_
```

（2）在进行计算前，显示数组 M 开始的 n+2 个字的内存值的截图（只能显示这 n+2 个字的内存值，多显示、少显示均扣分）

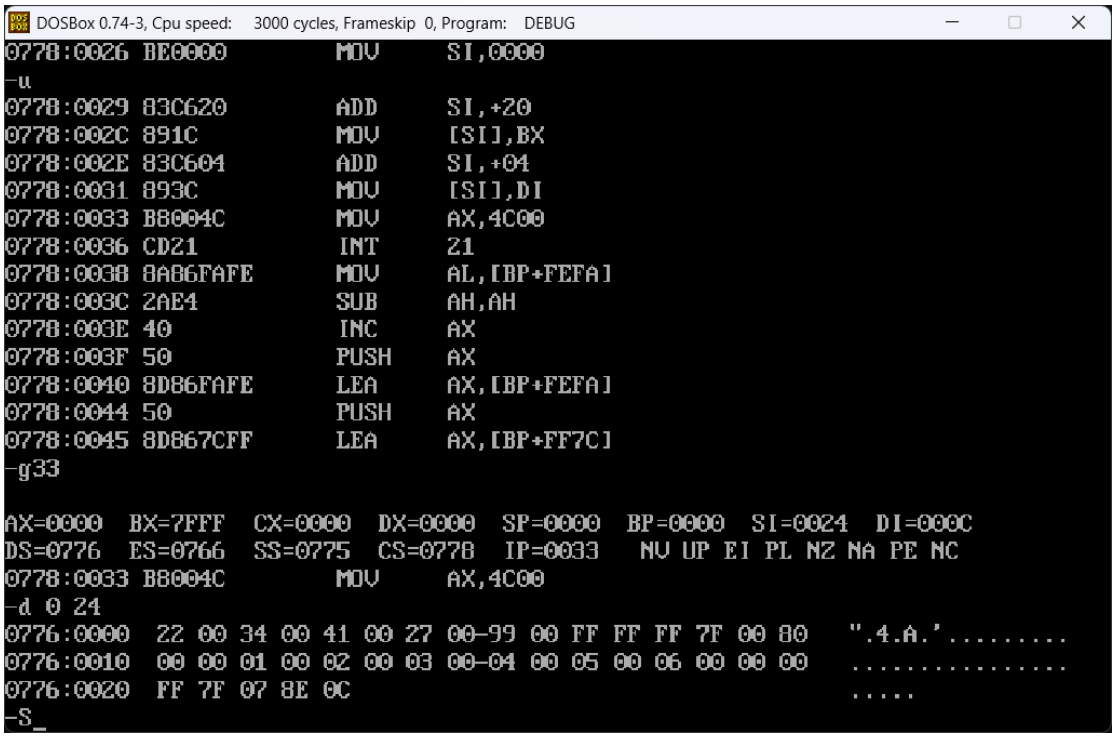
```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0778:0038 8A86FAFE      MOV     AL,[BP+FEFA]
0778:003C 2AE4          SUB     AH,AH
0778:003E 40            INC     AX
0778:003F 50            PUSH    AX
0778:0040 8D86FAFE      LEA     AX,[BP+FEFA]
-t

AX=0776  BX=0000  CX=0058  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=0766  ES=0766  SS=0775  CS=0778  IP=0003   NV UP EI PL NZ NA PO NC
0778:0003 8ED8          MOV     DS,AX
-t

AX=0776  BX=0000  CX=0058  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=0776  ES=0766  SS=0775  CS=0778  IP=0005   NV UP EI PL NZ NA PO NC
0778:0005 BE0000        MOV     SI,0000
-t

AX=0776  BX=0000  CX=0058  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=0776  ES=0766  SS=0775  CS=0778  IP=0008   NV UP EI PL NZ NA PO NC
0778:0008 B91000        MOV     CX,0010
-d 0 24
0776:0000  22 00 34 00 41 00 27 00-99 00 FF FF FF 7F 00 80   ".4.A.'.........
0776:0010  00 00 01 00 02 00 03 00-04 00 05 00 06 00 00 00   ................
0776:0020  B8 76 07 8E D8                                    .v...
-S
```

（3）执行完计算后，显示数组 M 开始的 n+2 个字的内存值的截图（只能显示这 n+2 个字的内存值，多显示、少显示均扣分）



（4）源代码

```
    ; loop.asm program

title I love asm

data segment
    ; Assuming student ID: 2234412799
    ; First 5 words: 22H, 34H, 41H, 27H, 99H
    ; n = 16, so 16 words in M
    M dw 22H, 34H, 41H, 27H, 99H, 0FFFFH, 7FFFH, 8000H, 0, 1, 2,
3, 4, 5, 6
    ; M+32 (2n) will hold the max absolute value
    ; M+36 (2(n+1)) will hold the offset address
data ends

code segment
    assume cs:code, ds:data
    main    proc
```

```
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax

        ; TODO ...
        ; | add your code between arrows |
        ; v ---------------------------- v

        ; Initialize
        mov si, offset M  ; SI points to start of M
        mov cx, 16        ; n = 16
        mov bx, 0         ; BX will hold max absolute value
        mov di, 0         ; DI will hold offset of max

        ; Loop to find max absolute value
find_max:
        mov ax, [si]      ; Load current word
        ; Compute absolute value
        test ax, ax       ; Check if negative
        jns positive
        neg ax           ; If negative, negate
positive:
        cmp ax, bx        ; Compare with current max
        jle not_greater
        mov bx, ax        ; Update max
        mov di, si        ; Update offset
not_greater:
        add si, 2         ; Next word
        loop find_max
```

```
        ; Store max value at M+32
        mov si, offset M
        add si, 32
        mov [si], bx

        ; Store offset at M+36
        add si, 4
        mov [si], di

        ; ^ ---------------------------- ^
        ; |         The END            |

        ; method 2: return to dos
        mov   ax, 4c00h
        int   21h
    main    endp
code ends
end main
```

2、 分支程序设计

（1）反汇编的截图

```
DOSBox 0.74-3, Cpu speed:    3000 cycles, Frameskip  0, Program:    DEBUG
0779:0030 8A04          MOV     AL,[SI]
0779:0032 3AC3          CMP     AL,BL
0779:0034 7706          JA      003C
0779:0036 7208          JB      0040
0779:0038 3AD7          CMP     DL,BH
0779:003A 7604          JBE     0040
0779:003C 8AD8          MOV     BL,AL
0779:003E 8AFA          MOV     BH,DL
0779:0040 46            INC     SI
-u 0
0779:0000 BB7607        MOV     AX,0776
0779:0003 8ED8          MOV     DS,AX
0779:0005 8D360000      LEA     SI,[0000]
0779:0009 B91800        MOV     CX,0018
0779:000C 8A04          MOV     AL,[SI]
0779:000E 3C30          CMP     AL,30
0779:0010 720F          JB      0021
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-S
```

```
DOSBox 0.74-3, Cpu speed:    3000 cycles, Frameskip  0, Program:    DEBUG
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-u
0779:0021 46            INC     SI
0779:0022 E2E8          LOOP    000C
0779:0024 BE1800        MOV     SI,0018
0779:0027 B90A00        MOV     CX,000A
0779:002A 32DB          XOR     BL,BL
0779:002C 32FF          XOR     BH,BH
0779:002E 32D2          XOR     DL,DL
0779:0030 8A04          MOV     AL,[SI]
0779:0032 3AC3          CMP     AL,BL
0779:0034 7706          JA      003C
0779:0036 7208          JB      0040
0779:0038 3AD7          CMP     DL,BH
0779:003A 7604          JBE     0040
0779:003C 8AD8          MOV     BL,AL
0779:003E 8AFA          MOV     BH,DL
0779:0040 46            INC     SI
-S_
```

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:0034 7706          JA      003C
0779:0036 7208          JB      0040
0779:0038 3AD7          CMP     DL,BH
0779:003A 7604          JBE     0040
0779:003C 8AD8          MOV     BL,AL
0779:003E 8AFA          MOV     BH,DL
0779:0040 46            INC     SI
-u
0779:0041 FEC2          INC     DL
0779:0043 E2EB          LOOP    0030
0779:0045 8AD7          MOV     DL,BH
0779:0047 80C230        ADD     DL,30
0779:004A B402          MOV     AH,02
0779:004C CD21          INT     21
0779:004E B22C          MOV     DL,2C
0779:0050 B402          MOV     AH,02
0779:0052 CD21          INT     21
0779:0054 8AC3          MOV     AL,BL
0779:0056 32E4          XOR     AH,AH
0779:0058 B10A          MOV     CL,0A
0779:005A F6F1          DIV     CL
0779:005C 3C00          CMP     AL,00
0779:005E 7408          JZ      0068
0779:0060 0430          ADD     AL,30
-S
```

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:0050 B402          MOV     AH,02
0779:0052 CD21          INT     21
0779:0054 8AC3          MOV     AL,BL
0779:0056 32E4          XOR     AH,AH
0779:0058 B10A          MOV     CL,0A
0779:005A F6F1          DIV     CL
0779:005C 3C00          CMP     AL,00
0779:005E 7408          JZ      0068
0779:0060 0430          ADD     AL,30
-u
0779:0062 8AD0          MOV     DL,AL
0779:0064 B402          MOV     AH,02
0779:0066 CD21          INT     21
0779:0068 8AD4          MOV     DL,AH
0779:006A 80C230        ADD     DL,30
0779:006D B402          MOV     AH,02
0779:006F CD21          INT     21
0779:0071 B8004C        MOV     AX,4C00
0779:0074 CD21          INT     21
0779:0076 827DFF00      CMP     BYTE PTR [DI-01],00
0779:007A 2BC0          SUB     AX,AX
0779:007C 50            PUSH    AX
0779:007D 8D867DFF      LEA     AX,[BP+FF7D]
0779:0081 50            PUSH    AX
-S_
```

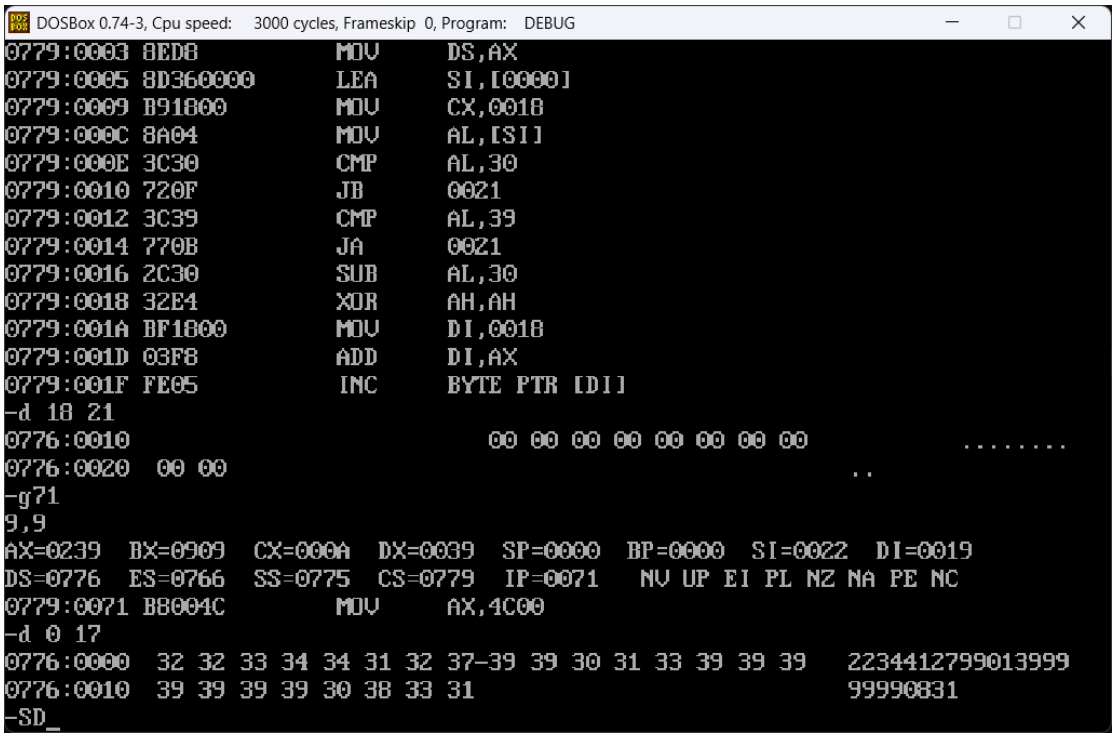
（2）在进行计算前，显示在数据段中定义的含学号的字符串的内存值的截图（只能显示该完整的字符串，多显示、少显示均扣分）

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
0779:0021 46            INC     SI
0779:0022 E2E8          LOOP    000C
0779:0024 BE1800        MOV     SI,0018
0779:0027 B90A00        MOV     CX,000A
-u 0
0779:0000 B87607        MOV     AX,0776
0779:0003 8ED8          MOV     DS,AX
0779:0005 8D360000      LEA     SI,[0000]
0779:0009 B91800        MOV     CX,0018
0779:000C 8A04          MOV     AL,[SI]
0779:000E 3C30          CMP     AL,30
0779:0010 720F          JB      0021
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-d 0 17
0776:0000  32 32 33 34 34 31 32 37-39 39 30 31 33 39 39 39   2234412799013999
0776:0010  39 39 39 39 30 38 33 31                           99990831
-SD
```

（3）在进行计算前，显示在数据段中定义的 COUNT 数组的内存值的截图（只能显示完整的 COUNT 数组内容，多显示、少显示均扣分）

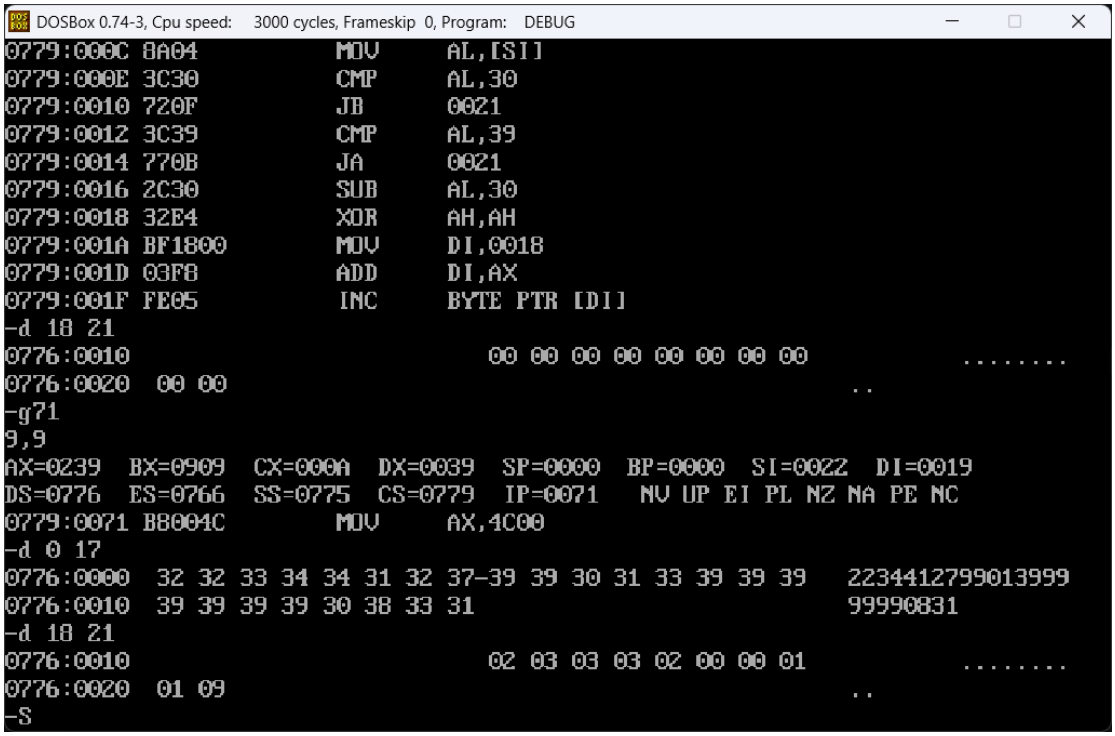
```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:0036 7208          JB      0040
0779:0038 3AD7          CMP     DL,BH
0779:003A 7604          JBE     0040
0779:003C 8AD8          MOV     BL,AL
0779:003E 8AFA          MOV     BH,DL
0779:0040 46            INC     SI
-u 0
0779:0000 B87607        MOV     AX,0776
0779:0003 8ED8          MOV     DS,AX
0779:0005 8D360000      LEA     SI,[0000]
0779:0009 B91800        MOV     CX,0018
0779:000C 8A04          MOV     AL,[SI]
0779:000E 3C30          CMP     AL,30
0779:0010 720F          JB      0021
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-d 18 21
0776:0010                     00 00 00 00 00 00 00 00          ........
0776:0020  00 00                                              ..
-S
```

（4）执行完计算后，显示在数据段中定义的含学号的字符串的内存值的截图（只能显示该完整的字符串，多显示、少显示均扣分）



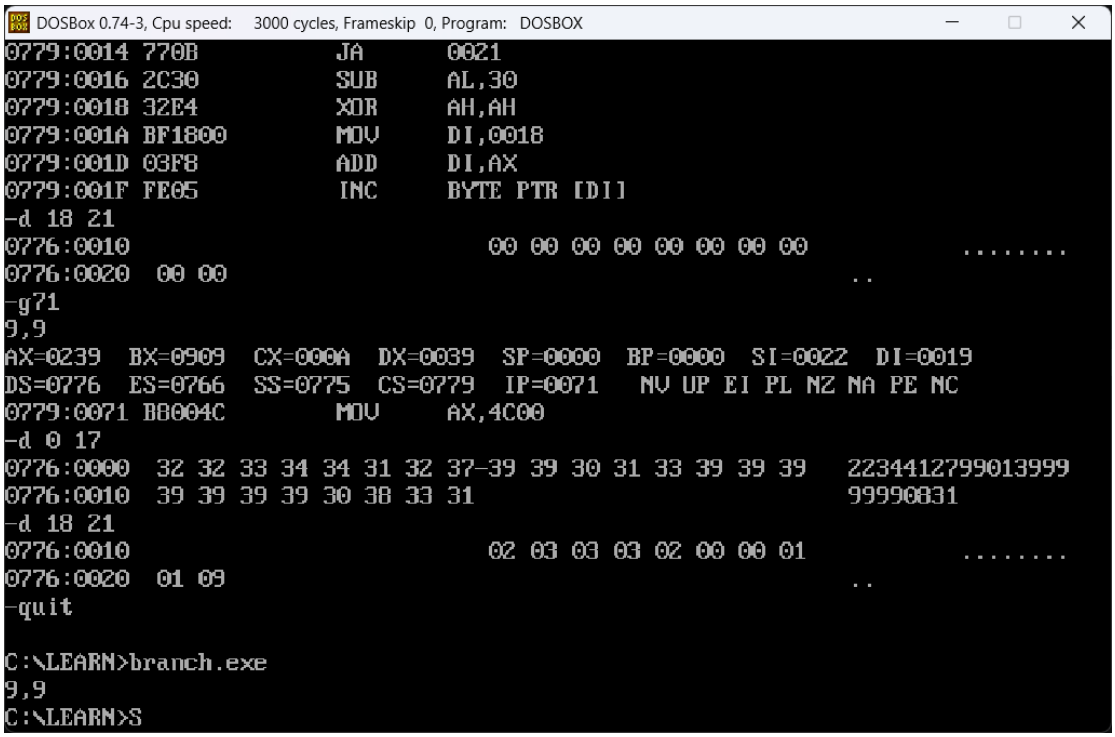
```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:0003 8ED8          MOV     DS,AX
0779:0005 8D360000      LEA     SI,[0000]
0779:0009 B91800        MOV     CX,0018
0779:000C 8A04          MOV     AL,[SI]
0779:000E 3C30          CMP     AL,30
0779:0010 720F          JB      0021
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-d 18 21
0776:0010                          00 00 00 00 00 00 00 00          ........
0776:0020  00 00                                                    ..
-g71
9,9
AX=0239  BX=0909  CX=000A  DX=0039  SP=0000  BP=0000  SI=0022  DI=0019
DS=0776  ES=0766  SS=0775  CS=0779  IP=0071   NV UP EI PL NZ NA PE NC
0779:0071 B8004C        MOV     AX,4C00
-d 0 17
0776:0000  32 32 33 34 34 31 32 37-39 39 30 31 33 39 39 39   2234412799013999
0776:0010  39 39 39 39 30 38 33 31                            99990831
-SD_
```

（5）执行完计算后，显示在数据段中定义的 COUNT 数组的内存值的截图（只能显示完整的 COUNT 数组内容，多显示、少显示均扣分）



```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0779:000C 8A04          MOV     AL,[SI]
0779:000E 3C30          CMP     AL,30
0779:0010 720F          JB      0021
0779:0012 3C39          CMP     AL,39
0779:0014 770B          JA      0021
0779:0016 2C30          SUB     AL,30
0779:0018 32E4          XOR     AH,AH
0779:001A BF1800        MOV     DI,0018
0779:001D 03F8          ADD     DI,AX
0779:001F FE05          INC     BYTE PTR [DI]
-d 18 21
0776:0010                          00 00 00 00 00 00 00 00          ........
0776:0020  00 00                                                    ..
-g71
9,9
AX=0239  BX=0909  CX=000A  DX=0039  SP=0000  BP=0000  SI=0022  DI=0019
DS=0776  ES=0766  SS=0775  CS=0779  IP=0071   NV UP EI PL NZ NA PE NC
0779:0071 B8004C        MOV     AX,4C00
-d 0 17
0776:0000  32 32 33 34 34 31 32 37-39 39 30 31 33 39 39 39   2234412799013999
0776:0010  39 39 39 39 30 38 33 31                            99990831
-d 18 21
0776:0010                          02 03 03 03 02 00 00 01          ........
0776:0020  01 09                                                    ..
-S
```

（6）程序在 DOSBox 下直接运行的截图



（7）源代码

```
; empty asm file

title I love asm

data segment
    ; Student ID based string (length > 20)
    STR    db  '22344127990139999990831'
    STRLEN equ $ - STR

      ;  COUNT[0]..COUNT[9]  store  occurrences  for  character
'0'..'9'
    COUNT  db  10 dup(0)
data ends

code segment
    assume cs:code, ds:data
    main    proc
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax

        ; TODO ...
        ; | add your code between arrows |
        ; v ---------------------------- v
        ; Count each digit's occurrences into COUNT array
```

```
    lea   si, STR
    mov   cx, STRLEN
count_loop:
    mov   al, [si]
    cmp   al, '0'
    jb    next_char
    cmp   al, '9'
    ja    next_char
    sub   al, '0'
    xor   ah, ah
    mov   di, offset COUNT
    add   di, ax
    inc   byte ptr [di]
next_char:
    inc   si
    loop  count_loop

    ; Find the digit with maximum count.
    ; If counts are equal, keep the larger digit.
    mov   si, offset COUNT
    mov   cx, 10
    xor   bl, bl        ; max count
    xor   bh, bh        ; max digit (0..9)
    xor   dl, dl        ; current digit index
find_max:
    mov   al, [si]
    cmp   al, bl
    ja    update_max
    jb    skip_update
    cmp   dl, bh
    jbe   skip_update
update_max:
    mov   bl, al
    mov   bh, dl
skip_update:
    inc   si
    inc   dl
    loop  find_max

    ; Output format: digit,count   (example: 9,9)
    mov   dl, bh
    add   dl, '0'
    mov   ah, 02h
    int   21h
```

```
        mov   dl, ','
        mov   ah, 02h
        int   21h

        ; Print count in decimal (0..99)
        mov   al, bl
        xor   ah, ah
        mov   cl, 10
        div   cl           ; AL = tens, AH = ones
        cmp   al, 0
        je    print_ones
        add   al, '0'
        mov   dl, al
        mov   ah, 02h
        int   21h
    print_ones:
        mov   dl, ah
        add   dl, '0'
        mov   ah, 02h
        int   21h

        ; ^ ---------------------------- ^
        ; |         The END            |

        ; method 2: return to dos
        mov   ax, 4c00h
        int   21h
    main    endp
code ends
end main
```